Politecnico di Milano

Ingegneria del Software – a.a. 2008/09

Appello del 11 settembre 2008

Cognome
Nome
Matricola
Sezione (segnarne una) Baresi □, Ghezzi □, San Pietro □

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Esercizio 4: .....

Totale .....

Esercizio 1 (punti 7)

Si consideri la seguente specifica dell'ADT Audio. Questo è un tipo immutabile che rappresenta un clip audio (p.es. brano musicale) codificato in digitale. Un tipico Audio è una quadrupla [codec, durata, traccia e bitRateMedio], dove "codec" è una stringa che descrive il codec utilizzato per la codifica del brano, "durata" è la durata in secondi del brano, "traccia" è la rappresentazione digitale (un array di byte) della musica del brano e "bitRateMedio" è il numero medio di Kbit al secondo della codifica, pari al numero di byte diviso la durata.

```
public /*@ pure */@class Audio {

        //costruttore:
        //@ensures(*crea un nuovo Audio sulla base dei valori passati come parametri*);
        public Audio(String codec, int durata, byte[] traccia);

        //metodi observer:
        //@ensures(*restituisce il  codec, autore e titolo*);
        public String codec();

        //@ensures(*restituisce la durata in secondi del brano*);
        public int durata();

        //@ensures(*restituisce la rappresentazione digitale dell'Audio *);
        public byte[] traccia ();

        //restituisce il bitrate medio del brano:
        //@ensures \result  == traccia().length / durata();
        public int bitRateMedio();
}
```

a- Si consideri anche l'ADT Video. Questo ha gli stessi metodi di Audio, ma tiene conto anche della traccia Video e quindi aggiunge i metodi seguenti:

```
public class Video {

        //@ensures(*restituisce la rappresentazione digitale dell'Audio *);
        public byte[] tracciaVideo ();


        //@ensures(*crea un nuovo clip video sulla base dei valori passati come parametri*);
        public Video(String info, int durata, byte[]tracciaAudio, byte[] tracciaVideo);

        //e ridefinisce il seguente:
        //@ ensures bitrate()*durata == traccia().length + tracciaVideo().length;
        public int bitRateMedio();
```

È possibile definire l'ADT MP3 come estensione di Audio, soddisfacendo il principio di sostituzione di Liskov? Giustificare la risposta.

L'ADT non puo' soddisfare il principio di sostituzione, in quanto occorre ridefinire il metodo bitrateMedio con una postcondizione che *non* implica la postcondizione originale.

b- Si consideri ora l'ADT MP3, che ha gli stessi metodi di Audio con stessa specifica, salvo per la seguente definizione del metodo codec:

public class MP3 {

//@\result == "MPEG3";
public String codec() {}

MP3 ha inoltre il seguente costruttore:

//@ensures(*crea un nuovo MP3di sulla base dei valori passati come parametri*);
public MP3(int durata, byte[] traccia);

È possibile definire l'ADT MP3 come estensione di Audio, soddisfacendo il principio di sostituzione di Liskov? Giustificare la risposta.

Si, in quanto la postcondizione di codec() e' piu' forte di quella originale. Non sono inoltre violate proprieta' di Audio, perche' certo MPEG3 era un codec valido anche per Audio.

Esercizio 2 (punti 8)

a- Si consideri la specifica della seguente classe Supporto, che rappresenta una memoria per clip audio (es. CD, Scheda SD, lettore MP3 ecc.). Supporto è un tipo mutabile. Un supporto contiene elementi di tipo Audio. Un Supporto tipico è identificato dalla coppia [CAPIENZA, brani], dove; "CAPIENZA" è una costante (per ogni supporto) che rappresenta la capacità in byte del supporto, "brani" è una sequenza <b1 b2 … bn> di oggetti della classe Audio. Descrive la post-condizione dei metodi spazioDisponibile e inserisci.

```
public class Supporto{

    public final int CAPIENZA;

    //@ensures (* /result contiene tutti e soli i brani di this, nell'ordine di memorizzazione *);
    public ArrayList<Audio> brani() {}

    //@requires max_capienza>0;
    //@ensures (* crea una nuova Memoria di capienza specificata, inizialmente vuota*)
    public Supporto(int max_capienza){

    //restituisce true  se nella Memoria this c'è ancora abbastanza spazio per il brano b
    //@ensures
```

CAPIENZA>= b.traccia().length +
(\sumof int i; 0<=i && i<brani().size(); brani().get(i).traccia().length))

```
    public boolean spazioDisponibile(Audio b) {

    //inserisce il brano b in this
    //@requires spazioDisponibile(b);
    //@ensures
```

brani().size() == \old(brani().size()+1) && b== brani().lastElement() &&
(\forall int i; 0<=i && i<brani().size()-1; brani().get(i) == \old(brani().get(i)))

```
    public void inserisci (Audio b) {}
```

}

b- È dato il seguente rep per la classe Supporto.

private ArrayList<BranoMusicale> contenuto;
private int durata; //durata complessiva dei brani memorizzati in contenuto.

e la seguente implementazione del costruttore:

```
public Supporto(int max_capienza){
        CAPIENZA=max_capienza;
        this.contenuto=new ArrayList<Audio>();
        this.durata=0;
}
```

Descrivere l'invariante privato e implementare i metodi seguenti:

//@ private invariant

contenuto!=null && contenuto.size() == brani().size() &&
durata == (\sumof int i; 0<=i && i<contenuto.size(); contenuto.get(i).getDurata()) &&
(\forall i; 0<=i && i<brani().size(); brani().get(i)==contenuto.get(i);
(NB: si suppone una relazione uno a uno fra la posizione nel metodo *brani()* e nella variabile d'istanza *contenuto*)

public ArrayList<Audio> brani() {

}

return contenuto.clone()

public boolean spazioDisponibile(Audio b) {

int somma =0;
(for int i; i>0 && i< contenuto.size(); i++)
somma+=contenuto.getMusica().length;.
return (b.traccia().length>=capienza -somma);.
}

public void inserisci (Audio b) { contenuto.add(b);

```
        }
}
```

c- Con riferimento al punto (b) dell'esercizio 1, si consideri la classe MP3Memory, che definisce una memoria contenente solo brani di tipo MP3.

```
public class MP3Memory extends Supporto{
//@also
//@public invariant
```

(\forall int i; 0<=i && i<brani().size(); brani().get(i) instanceof MP3);

```
        //@requires max_capienza>0;
        //@ensures (* crea una nuova MP3Memory, di capienza specificata, inizialmente vuota*)
        public MP3Memory(int max_capienza) {}
}
```

Si completi il public invariant e si indichi, motivando accuratamente ma sinteticamente la risposta, se questo ADT soddisfa o meno il principio di sostituzione di Liskov rispetto a Supporto.

L'ADT (un contenitore) non soddisfa il principio di sostituzione (anche se la versione data soddisfa la regola dei metodi), in quanto il suo invariante deve impedire di fatto l'inserimento di brani che non siano di tipo MP3. Per soddisfare l'overview (e quindi l'invariante) occorrerebbe allora ridefinire la specifica del metodo inserisci, per impedire l'inserimento di Audio che non siano MP3. Questo tuttavia violerebbe la regola della postcondizione.

Esercizio 3 (punti 6)

Supponendo che esistano le classi Mia, Tua e Sua, dove Mia è super classe sia di Tua che di Sua, quali delle seguenti affermazioni sono corrette?

1. Tua e Sua ereditano i costruttori di Mia

SCORRETTA

2. Se Mia definisce un attributo x privato, qualunque metodo di Tua e Sua potrebbe leggerne e scriverne il valore direttamente

SCORRETTA

3. Scrivere: Mia m;
   m = new Mia();
   darebbe un errore in compilazione

SCORRETTA

4. Scrivere: Mia m;
   m = new Tua();
   darebbe un errore in compilazione

SCORRETTA

5. Scrivere: Tua m;
   m = new Mia();
   darebbe un errore in compilazione

CORRETTA

6. Scrivere: Tua m;
   m = new Sua();
   darebbe un errore in compilazione

CORRETTA

7. Se Mia, Tua e Sua fossero definite in tre package diversi, ci sarebbe almeno un errore in compilazione

SCORRETTA

8. Se Mia definisce un attributo x 'friendly', qualunque metodo di Tua e Sua (definite in package diversi rispetto a Mia) potrebbe leggerne e scriverne il valore direttamente

SCORRETTA

Esercizio 4 (punti 4)

Un distributore automatico di merendine è composto da un display, una tastiera, una gettoniera e un distributore vero e proprio. Questi elementi hardware sono controllati da software opportuno per consentire all'utente di scegliere un prodotto, pagare con il proprio dispositivo attivo (chiavetta), o in contanti, e recuperare il prodotto acquistato.
Chiaramente, ogni prodotto ha un prezzo, leggermente inferiore se il cliente paga con la propria chiavetta, e il distributore non eroga nulla se la cifra pagata non è sufficiente (il credito sul dispositivo non è sufficiente, oppure le monete inserite sono troppo poche). Se il totale delle monete inserite è superiore al prezzo richiesto, la machina, attraverso la gettoniera, deve dare il resto. Il cliente può anche usare la gettoniera per caricare la propria chiavetta; questo avviene inserendo soldi (monete e banconote) senza selezionare un prodotto.

a- Si modelli il sistema attraverso un opportuno diagramma delle classi UML, specificando quali classi "rappresentano" elementi puramente software e quali definiscono l'interfaccia dei componenti hardware elencati in precedenza.

b- Si scriva la specifica JML del metodo definito per la selezione di un prodotto. Considerando che il codice scelto deve esistere (altrimenti il sistema deve prevedere un opportuno messaggio d'errore), il prodotto non deve essere esaurito e l'ammontare pagato sufficiente. Se il cliente paga con la chiavetta, il costo deve essere detratto dal credito del cliente, se paga in contanti, si deve prevedere l'eventuale resto.